

School Management App

Library & Transport Modulok

React 18 + Laravel 12 API | 2026

RÉSZ 1 – Library (Könyvtár) Modul

1. A Library modul felépítése

A Library modul két entitást kezel: könyvek (LibraryBook) és kölcsönzések (LibraryIssue). Egy könyv kölcsönözhető diáknak VAGY tanárnak, és visszahozható. A modul három fő oldalból áll: könyvek listája (LibraryBookListPage), könyv form (LibraryBookFormPage), és kölcsönzések listája (IssuesListPage – aktív és lejárt egyaránt).

Komponens	Funkció	Minta
LibraryBookListPage	Könyvek listázása, elérhető szűrő	Táblázat + checkbox szűrő + IssueBookModal
LibraryBookFormPage	Könyv CRUD form	Szokásos CRUD minta (mint Students)
IssueBookModal	Kölcsönzés diáknak VAGY tanárnak	Modal, toggle gombokkal (diák/tanár váltás)
IssuesListPage	Aktív ÉS lejárt kölcsönzések	Egy komponens, :type URL paraméterrel

2. A Library modul elnevezési hibái – tanulságok

A Library modul fejlesztése során a legtöbb elnevezési hiba halmozódott fel az egész projektben. Ezek részletes dokumentálása fontos, hogy a jövőben elkerülhetők legyenek.

Hibás névalak	Helyes névalak	Hol fordult elő
class Library_book	class LibraryBook	LibraryBook.php modell osztálynév
class Library_issues	class LibraryIssue	LibraryIssue.php modell osztálynév
libraryIssues()	issues()	LibraryBook modell kapcsolat metódus neve
Library_issues::class	LibraryIssue::class	LibraryBook modell hasMany() hivatkozás

issues_date	issue_date	Migráció, fillable, casts, Resource
\$book->name	\$book->title	LibraryIssueResource book betöltésénél
unique:libraryBooks	unique:library_books	StoreLibraryBookRequest validáció
Schema::create('libraryBooks')	Schema::create('library_books')	Migrációs fájl

Mi okozta ezt az elnevezési káoszt?

A Library modul tábla- és osztálynevei kezdetben következetlenül lettek megadva: a PHP modell osztályok nevei (Library_book, Library_issues) nem egyeztek a Laravel PascalCase konvencióval (LibraryBook, LibraryIssue), a migrációs fájlokban camelCase tábla nevek szerepeltek (libraryBooks) a kötelező snake_case helyett (library_books), és a mezőnév (issues_date vs issue_date) is elgépelésből eredt. Ezek kaszkádszerűen okoztak hibákat egymás után, mert minden réteg (modell, migráció, resource, request) más-más kombinációt 'örökölt' az előző hibájából.

3. IssueBookModal – diák VAGY tanár kölcsönző

```
const [borrowerType, setBorrowerType] = useState('student');

// Payload összeállítása mentéskor:
const payload = {
  student_id: borrowerType === 'student' ? data.borrower_id : null,
  teacher_id: borrowerType === 'teacher' ? data.borrower_id : null,
  due_date: data.due_date,
};
```

Miért kell explicit null-t küldeni a másik mezőbe?

A backend IssueLibraryBookRequest validációja ellenőrzi hogy PONTOSAN az egyik legyen kitöltve (student_id VAGY teacher_id), nem mindkettő és nem egyik sem. Ha az el nem küldött mezőt egyáltalán nem szerepeltetnénk a payloadban (nem null, hanem hiányzó), a Laravel undefined-ként kezelné, ami nem egyezik a nullable validációs elvárásával. Az explicit null küldése egyértelmű jelzés: 'ez szándékosan üres'.

4. IssuesListPage – egy komponens, két mód

```
// Route definíció:
<Route path='/library/issues/:type' element={<IssuesListPage />} />
```

```
// A komponensben:
const { type } = useParams();
const isOverdue = type === 'overdue';

const activeQuery = useActiveIssues(page);
const overdueQuery = useOverdueIssues(page);

// A type alapján döntjük el melyik adatot mutatjuk:
const { data, isLoading, isError } = isOverdue ? overdueQuery :
activeQuery;
```

Miért hívjuk meg MINDKÉT hookot, ha csak az egyiket használjuk?

A React Hooks szabálya: hookokat nem lehet feltételesen meghívni (if-ben, switch-ben). Mindkét hookot meg kell hívni minden renderelésnél, de a FELHASZNÁLT adatot (data, isLoading, isError) feltételesen választjuk ki a hookozás UTÁN. Ez az egyetlen Rule of Hooks-ot tiszteletben tartó megközelítés dinamikus hook-választáshoz.

5. Kölcsönzés invalidálási stratégia

```
export function useIssueBook(bookId) {
  return useMutation({
    mutationFn: (data) => api.post(`/library/books/${bookId}/issue`,
data),
    onSuccess: () => {
      // MINDKÉT cache érvénytelen lesz:
      queryClient.invalidateQueries({ queryKey: ['library-books']
});
      queryClient.invalidateQueries({ queryKey: ['library-issues']
});
    },
  });
}
```

Miért invalidáljuk MINDKÉT cache-t?

Kölcsönzés után KÉT dolog változik egyszerre: 1) A könyv available_copies száma csökken (ezt a library-books cache-nek kell tükröznie, hogy az is_available badge helyesen jelenjen meg a könyv listában). 2) Egy új kölcsönzési bejegyzés jön létre (ezt a library-issues cache-nek kell tükröznie, hogy az aktív kölcsönzések listájában megjelenjen). Egyetlen invalidateQueries hívás mindkettőt frissíti a megfelelő prefix-egyezéssel.

RÉSZ 2 – Transport (Közlekedés) Modul

6. A Transport modul felépítése

A Transport modul a projekt legösszetettebb modulja: egy járáshoz megállók tartoznak (TransportStop), és diákok rendelhetők hozzá egy adott megállón keresztül (TransportStudent). Ez három különálló modellt, és egy részlet oldalt (TransportDetailPage) is igényel.

Komponens / Fájl	Funkció
TransportListPage	Járatok listázása, diák hozzárendelés modal, törlés
TransportFormPage	Járat CRUD + megállók dinamikus kezelése
TransportDetailPage	Egy járat részletei: megállók táblázat + diákok táblázat
AssignStudentModal	Diák hozzárendelése egy megállóhoz
useTransport.js	React Query hook-ok: lista, egy elem, CRUD, assign, remove

7. Dinamikus megálló kezelés a formban

A TransportFormPage az eddigi legbonyolultabb form: a megállót (stops) NEM React Hook Form-mal, hanem külön useState-tel kezeljük, mert dinamikusan hozzáadható/törölhető elemekről van szó.

7.1 Miért külön useState és nem React Hook Form?

```
const [stops, setStops] = useState([emptyStop()]);

function emptyStop(order = 1) {
  return { stop_name: '', pickup_time: '', drop_time: '', order };
}

function addStop() {
  setStops((current) => [...current, emptyStop(current.length + 1)]);
}

function removeStop(index) {
  setStops((current) =>
    current.length === 1 ? current // legalább 1 megálló kell
    : current.filter((_, i) => i !== index)
      .map((stop, i) => ({ ...stop, order: i + 1 })) // order
    újraszámolás
  );
}
```

Miért nem React Hook Form a megállókhöz?

A React Hook Form erőssége egyszerű, fix struktúrájú formoknál van (nevesített mezők, pl. 'name', 'email'). A megállók DINAMIKUS tömbök - nem tudni előre hány lesz belőlük. Bár a React Hook Form támogatja a useFieldArray() hook-ot tömbmezőkhöz, ez extra komplexitást jelent. Külön useState-tel a tömbkezelés egyszerűbb és olvashatóbb: addStop(), removeStop(), updateStop() - mindenki érti mi történik.

7.2 Megállók előkészítése a payload-ban

```
const payload = {
  ...data, // React Hook Form mezők (route_name, vehicle_number stb.)
  capacity: Number(data.capacity),
  stops: stops
  .filter((stop) => stop.stop_name.trim() !== '') // üres megállók
  kiszűrése
  .map((stop, index) => ({
    ...(stop.id ? { id: stop.id } : {}), // szerkesztésnél id
    megőrzése
    stop_name: stop.stop_name,
    pickup_time: stop.pickup_time || null,
    drop_time: stop.drop_time || null,
    order: index + 1,
  })),
};
```

A ...(stop.id ? { id: stop.id } : {}) minta

Szerkesztés módban a meglévő megállóknak VAN id-juk (az adatbázisból jöttek), az újonnan hozzáadottaknak NINCS. A backend TransportService::syncStops() metódusa ennek alapján dönti el: ha van id, FRISSÍTÍ a meglévő megállót; ha nincs id, LÉTREHOZZA az újat. A spread operátoros feltételes hozzáadás (...(stop.id ? {id} : {})) elkerüli hogy 'undefined' kerüljön az id mezőbe az új megállóknál.

8. TransportDetailPage – elkülönített részlet nézet

A Transport modul egyedi jellemzője hogy van egy KÜLÖN részlet oldala (TransportDetailPage) a szerkesztés oldal mellett. Ez azért szükséges mert a járat részletei (megállók táblázata + diákok táblázata) összetett, és a szerkesztő form már önmagában is bonyolult a megállók dinamikus kezelésével.

```
// Route-ok - a sorrend kritikus!
<Route path='/transports' element={<TransportListPage />} />
<Route path='/transports/create' element={<TransportFormPage />} />
<Route path='/transports/:id' element={<TransportDetailPage />} />
<Route path='/transports/:id/edit' element={<TransportFormPage />} />

// Miért működik a /create és /:id együtt?
```

```
// React Router v6 a SPECIFIKUSABB (statikus) route-ot preferálja
// ezért a /create nem ütközik a /:id-vel
```

9. AssignStudentModal – megálló és diák egyszerre

```
// A modal a transport objektumot kapja propként,
// beleértve a már betöltött stops tömböt is:
const stops = transport.stops ?? [];

// Ha nincs megálló, a submit gomb le van tiltva:
<button disabled={assignStudent.isPending || stops.length === 0}>

// A megállók listája a transportból jön (nem külön API hívás):
{stops.map((stop) => (
  <option key={stop.id} value={stop.id}>{stop.stop_name}</option>
))}
```

Miért nem kell külön API hívás a megállókhoz a modalban?

Amikor az AssignStudentModal megnyílik, a szülő komponens (TransportListPage vagy TransportDetailPage) MÁR betöltötte a transport adatát a useTransport() hookkal, beleértve a stops tömböt is (a backend with(['stops']) eager loadingot használ). A transport objektumot propként adjuk át a modalnak, így a megállók azonnal elérhetők - felesleges egy plusz API hívást indítani amit már egyszer lekérdeztünk.

10. is_full – számított mező a Resource-ban

```
// TransportResource.php
'is_full' => $this->when(
    isset($this->students_count),
    $this->students_count >= $this->capacity
),

// Frontend használat:
{!transport.is_full && (
  <button onClick={() => setAssignModalTransport(transport)}>
    Diák hozzárendelése
  </button>
)}
```

Miért whenCounted() és not közvetlen \$this->students_count?

A `whenCounted()` csak akkor adja vissza a `students_count` értéket, ha a lekérdezésben `withCount(['students'])` szerepelt. Ha valahol a kódban nem használták a `withCount()`-ot (pl. egy egyszerű `findOrFail()` hívásban), a `students_count` NULL lenne, és a `$this->students_count >= $this->capacity` összehasonlítás hibás eredményt adna. A `when(isset(...))` biztosítja hogy az `is_full` csak akkor számíttódik ki, ha a szükséges adat ténylegesen rendelkezésre áll.

11. Összesített hibakeresési napló – Library & Transport

Hiba	Ok	Megoldás
Library_book relationship issues hibák	Helytelen modell osztálynevek (snake_case vs PascalCase)	Minden modell átnevezése PascalCase-re
issues_date column not found	Migrációban és modellben eltérő mezőnév	issue_date egységesítése mindenhol
book->name undefined	LibraryBook modellnek title mezője van, nem name	Resource-ban title-re javítás
unique:libraryBooks validáció	camelCase tábla név a validációban	unique:library_books snake_case-re javítás
Transport stops dinamikus kezelés	React Hook Form nem ideális tömbhöz	Külön useState a megállókhoz

– School Management App fejlesztési dokumentáció –